# Cordaを用いた小口取引即時グロス銀行間決済システム

長塚 大翔[1,a)]　駒野 雄一[1,b)]

**概要**：全銀ネットは 2018 年に Hyperledger Fabric を用いた銀行間決済プラットフォームの実証実験を行った。しかし、Fabric を用いたシステムでは、全ノードと通信を行うため停止ノードが存在する場合にタイムアウト待ちが発生すること、及び、取引履歴を全銀行が共有する単一のデータベースに記録することから、縮退運転時におけるスループット性能の低下や、取引履歴増加に伴う残高取得のレスポンス性能の劣化の課題が生じた。本研究では、これらの課題を解決するために、ブロックチェーンプラットフォーム Corda を用いた小口取引即時グロス銀行間決済システムを提案し、簡易的なシステムを構築して性能を評価する。

**キーワード**：FinTech, ブロックチェーン, Corda

# A Real-Time Gross Settlement System for Small-Value Interbank Transactions Using Corda

Hiroto Nagatsuka[1,a)]　Yuichi Komano[1,b)]

***Abstract:*** Zengin Net conducted a proof-of-concept experiment for an interbank settlement platform using Hyperledger Fabric in 2018. However, the Fabric-based system faced several challenges. Because it requires communication with all nodes, timeout delays occurred when a node was down, leading to degraded system throughput during fail-soft operations. Furthermore, because transaction histories are recorded in a single database shared by all banks, response times for balance inquiries deteriorated as the transaction volume grew. Against these issues, this study proposes a retail real-time gross settlement interbank payment system using the Corda blockchain platform, constructs a simplified system, and conducts a performance evaluation.

***Keywords:*** FinTech, Blockchain, Corda

## 1. はじめに

近年、金融とテクノロジーが融合した FinTech が身近になり、ブロックチェーン技術が注目されている。ブロックチェーン技術は、耐改竄性や可用性の高さなどの特徴をもち、既にデジタル通貨やサプライチェーンなどで実用化への議論が進められている。

このような背景の下、日本国内の銀行間資金決済を担う一般社団法人全国銀行資金決済ネットワーク（全銀ネット）は、2018 年度にブロックチェーン実証実験タスクフォースを設置し、「ブロックチェーン技術を活用した新銀行間決済プラットフォーム実証実験」を行った [9]。この実証実験では、ブロックチェーン技術を用いた小口取引における RTGS（Real-Time Gross Settlement：即時グロス決済）の実現を目的として、銀行間資金決済専用のデジタル通貨（決済用デジタル通貨）を用いたシステムが構築された。システムの構築においては、基盤技術となるブロックチェーンプラットフォームとして Linux Foundation が運営する Hyperledger Project のひとつである Hyperledger Fabric（Fabric）を採用し、銀行間決済の基本機能が正常に動作することが確認された。

しかし、実証実験では、性能面に関して 2 点の課題が検出された。1 点目は、縮退運転時におけるスループット性能の低下である（課題 1）。システムの一部のノードが停止している場合、システム全体がタイムアウト待ちの影響を

[1] 千葉工業大学
Chiba Institute of Technology
[a)] s2232111jl@chibatech.ac.jp
[b)] yuichi.komano@chibatech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

受け、処理能力が低下することが判明した。具体的には、通常時のスループットが 746tps (transactions per second) であったのに対し、全種類のサーバを 1 台ずつ停止させた状態では 284tps まで低下し、約 62%の性能劣化が生じた。2 点目は、取引履歴の増加に伴う残高取得のレスポンス性能の劣化である（課題 2)。取引件数が増加するにつれて、決済用デジタル通貨の残高照会にかかる時間が増加する事象が確認された。測定データによれば、取引履歴が 0 件の時点では 43ms であったレスポンスが、約 6 万件の時点では 884ms、約 12 万件の時点では 1,724ms、約 18 万件の時点では 2,618ms と、データ量に比例して処理時間が増大した。これら 2 つの性能課題は、全ノードが World State とよばれる同一の台帳を共有するアーキテクチャと、ブロードキャスト型の通信方式を用いるという Fabric の特性に起因すると考えられる。

これらの課題を解決するために、本研究では Fabric とは異なるアーキテクチャをもつブロックチェーンプラットフォームである Corda を採用した、小口取引即時グロス銀行間決済システムを提案する。Corda は、取引に関係する当事者間でのみデータを共有する Need-to-know の原則に基づいた設計となっており、Fabric の特性に起因する 2 つの課題の解決が期待できる。

### 1.1 関連研究

ブロックチェーン技術を銀行間決済システムへ適用する試みは、国内外で行われている。

#### 1.1.1 全銀ネットによる実証実験

全銀ネットは、2018 年度に「ブロックチェーン技術を活用した新銀行間決済プラットフォーム実証実験」を実施した [9]。この実証実験では、Fabric を用いた小口取引の RTGS 実現可能性を検証し、銀行間決済の基本機能が正常に動作することが確認された。一方で、縮退運転時のスループット低下および取引履歴増加に伴うレスポンス性能の劣化という性能課題が明らかになった。この実証実験の詳細および課題の分析については、3 節で述べる。

#### 1.1.2 Project Jasper

海外では、Payments Canada とカナダ銀行が主導する「Project Jasper」が、DLT を用いた大口決済システムの実証実験を複数フェーズにわたって実施した [4]。Phase 1 では Ethereum を用いたシステムを構築したが、PoW アルゴリズムに起因する確率的ファイナリティの問題や、全ノードが全取引情報を共有することによるプライバシー確保の困難さが課題として浮上した。これらの課題を踏まえ、Phase 2 では Corda を採用し、即時かつ確定的なファイナリティの実現と、金融機関が求める厳格なプライバシー要件の充足を確認した。結論として、DLT 単体の導入によるコスト削減効果は現時点では限定的であるとしつつも、Corda のようなプライバシーとファイナリティを重視した設計が、金融市場インフラ全体の強靭性と効率性を向上させる可能性があると結論付けている。

本研究では、Project Jasper で示された Corda のアーキテクチャ上の優位性に着目し、全銀ネットの実証実験で明らかとなった性能課題の解決に応用する。

### 1.2 本論文の貢献

本研究の貢献は以下のとおりである。

- Corda を用いた小口取引即時グロス銀行間決済システムのアーキテクチャを提案する。
- Docker コンテナ上に Corda ネットワークを構築し、提案システムの基本機能を実装・検証する。
- RPC クライアントを用いた並列処理による TPS 測定および取引件数増加に伴う残高照会レスポンスの計測を行い、実証実験の結果と比較・考察する。
- 縮退運転時の可用性検証として、一部ノード停止時の取引継続性を確認する。

本論文の構成は以下のとおりである。2 節では準備として、Hyperledger Fabric と Corda について説明し、Corda の銀行システムへの活用検討事例を紹介する。3 節では全銀ネットの実証実験の概要とその問題点を述べる。4 節では提案システムを説明する。5 節では提案システムの実装と実験について報告する。6 節では実証実験との比較を含む考察を行う。7 節では結論と今後の課題を述べる。

## 2. 準備

本節では、ブロックチェーンプラットフォームの Hyperledger Fabric と Corda について説明する。また、Corda の銀行システムへの活用検討事例について紹介する。

### 2.1 Hyperledger Fabric

Hyperledger Fabric [1] は、Linux Foundation が主導するオープンソースのブロックチェーンプラットフォームであり、金融やサプライチェーンといったエンタープライズ領域での利用を目的に設計されている。

Fabric は、参加者が限定された許可型（Permissioned）のネットワークである点が特徴である。Bitcoin [7] や Ethereum [3] などに代表される自由参加型（Permissionless）のパブリックチェーンは、不特定多数のノードが参加するため、匿名の悪意あるノードを排除するために PoW（Proof of Work, [7]）などの計算コストの大きい確率的合意形成が必要となる。一方、Fabric は、認証局（CA）によって発行されたデジタル証明書を保持するノードのみネットワークに参加可能とすることで、Raft [8] などの計算コストを抑えた合意アルゴリズムが利用可能となり、金融システムの性能面の要求に対応がしやすい。

また、金融システムにおいて重要なのがファイナリティ（決済の確定性）である。パブリックチェーンは確率的合

© 2026 Information Processing Society of Japan

意形成を必要とする確率的ファイナリティであるが、許可型は信頼できるノード間での単純な承認プロセスを経て即時ファイナリティを提供する。これにより、二重支払いのリスクを排除でき、金融システムの安定性が保証される。

Fabric の台帳（Ledger）は、単一のファイルではなく、用途の異なる二つのデータ構造によって構成される。一つはブロックチェーン（トランザクションログ）である。これは全ての状態遷移の履歴を記録するイミュータブルな連結リストであり、監査証跡の役割を担う。もう一つは、World State である。これは台帳の現在の値のみを保持するデータベース（DB）であり、一般的に LevelDB [5] や CouchDB といった Key-Value ストアとして実装される。そして、Fabric のネットワークに参加するすべてのノードは、原則として同一の台帳を保持し、ネットワーク全体で単一の状態を共有するモデルとなっている。

### 2.2 Corda

Corda [2], [6] は、R3 社によって開発された分散台帳技術（DLT）であり、特に金融機関などの企業間取引での利用を目的に設計されている。本論文においては、分散台帳技術についてブロックチェーン技術の一つとして表記する。

Corda は Fabric と同じく、参加者が限定された許可型のネットワークである。

また、Hearn[6] らによると、Corda は従来のブロックチェーンが抱えるプライバシーとスケーラビリティの課題を解決するため、データを全ノードにブロードキャストしない独自のアーキテクチャを採用していることが示されている。Bitcoin や Fabric などがネットワーク上の全ノードで同一の台帳を共有するのに対し、Corda は、取引に関係のあるノード間でのみデータを共有する Need-to-know の原則に沿った台帳となっている。例えば、A 銀行と B 銀行の間で行われる取引データは、A 銀行と B 銀行のノード、および必要であれば Notary のみに保存され、取引に関与しない C 銀行には共有されない。つまり、トランザクションの内容を全ノードにブロードキャストする必要はない。各ノードは Vault とよばれるローカルの DB を持ち、部分的に重なり合う事実を共有することで全体としての整合性を保っている。これにより、データのプライバシー保護と、ネットワーク全体への負荷分散が実現している。

データモデルは、Bitcoin と同様に UTXO（Unspent Transaction Output）モデルを拡張した設計となっている。台帳上の事実は State という不変のオブジェクトとして表現され、トランザクションは既存の State を入力として消費し、新たな State を出力として生成することで台帳を更新する。各トランザクションは、入力となる過去のデータのハッシュ値を含んでおり、仮に過去のデータが 1 ビットでも改竄されると、そのハッシュ値が変わることで、それ以降のすべてのトランザクションのハッシュ値も整合性が取れなくなる。つまり、ある時点のデータを改竄するためには、そこから派生した全ての歴史的トランザクションを書き換える必要があり、計算論的に不可能である。

### 2.3 Corda を活用した銀行システム

2016 年、Payments Canada、カナダ銀行、R3、および R3 コンソーシアムに加盟するカナダの商業銀行が共同して、DLT をベースにした大口決済システムを検討するための実証実験「Project Jasper」を行った [4]。この実証実験は、DLT が現在の中央銀行システムと比較して、決済の効率性、安全性、およびコスト削減に寄与するかを評価することを目的とした。プロジェクトは複数のフェーズに分けて実施され、各フェーズで異なるプラットフォームの検証が行われた。

初期の Phase 1 では、パブリックブロックチェーンの Ethereum を用いたシステム構築が行われた。しかし、この検証を通じて、PoW アルゴリズムに起因するファイナリティの性質が懸念点となることが判明した。また、Ethereum の標準的な仕様では全ノードが全ての取引情報を共有するため、銀行間の取引における顧客情報や取引額の秘匿性確保が困難であることや、全ノード共有によるスケーラビリティの限界も課題として浮上した。

これらの課題を踏まえ、Phase 2 ではプラットフォームを Corda に変更して検証が進められた。Corda のデータを全ノードにブロードキャストしないアーキテクチャにより、金融機関が求める厳格なプライバシー要件を満たしつつ、特定の Notary による一意性の検証を通じて、即時かつ確定的なファイナリティを実現できることが確認された。

結論として、DLT 単体の導入によるバックオフィスのコスト削減効果は現時点では限定的であるとしつつも、Corda のようなプライバシーとファイナリティを重視した設計を採用することで、金融市場インフラ全体の強靭性と効率性を向上させる可能性があると結論付けている。

本研究では、この Project Jasper で導き出された金融システムにおける Corda のアーキテクチャ上の優位性に着目し、全銀ネットの実証実験の課題解決に応用する。

## 3. 全銀ネット実証実験とその問題点

本節では、全銀ネットが 2018 年度に実施した「ブロックチェーン技術を活用した新銀行間決済プラットフォーム実証実験」について概説する。実験の目的、環境、方法、および結果を整理し、そこで検出された性能課題を把握する。そして、それらの課題に対して本研究が提案する Corda を用いたアプローチがどのように有効であるかを考察する。

### 3.1 実証実験の概要

実証実験では、ブロックチェーン技術が銀行間資金決済、特に小口取引における RTGS の基盤として有用であるかを

© 2026 Information Processing Society of Japan

検証することを目的とした。検証に際し、銀行間資金決済専用のデジタル通貨をシステム上で発行・流通・回収する、「新銀行間決済プラットフォーム」の構築を目指した。また、銀行間決済のトリガーとなる送金取引を発生させるため、メガバンク 3 行および富士通が開発した個人間送金システム（P2P 送金 PF）を活用し、これと連動させることで、送金発生から銀行間決済までの一連の流れを一気通貫で検証した。基盤となるブロックチェーンプラットフォームには、Hyperledger Fabric が採用された。システム環境は、可用性の検証を十分に行うため、すべてのサーバ種別において冗長化構成がとられた。

検証は大きく機能検証と非機能検証に分けて実施された。

機能検証では、決済用デジタル通貨の発行、銀行間送金、回収といった基本機能が正しく動作するか、および残高アラートや取引停止閾値といった決済リスク回避の仕組みが機能するかを確認した。また、日次で決済用デジタル通貨の残高整合性と取引履歴の完全性を確認し、RTGS として矛盾なく稼働しているかを検証した。

非機能検証では、「可用性」「性能」「セキュリティ」の 3 点に重点が置かれた。可用性においては、サーバの単一障害や複数障害が発生した場合でも業務が継続できるかを検証した。性能においては、目標とするスループット性能のピーク時 1,000tps（transactions per second）が発揮できるか、および取引量が増加した場合のレスポンス性能を測定した。セキュリティにおいては、ブロックチェーンの耐改竄性を評価するため、データへの直接アクセスによる改竄試行とその検知・復旧の挙動を確認した。

その結果、機能検証に関しては、予定された検証項目を全て消化し、決済用デジタル通貨を用いた RTGS としての基本機能が正常に動作することが確認された。P2P 送金 PF を含めたシステム全体としても、期間中にシステム停止することなく安定稼働した。

また、非機能検証に関しては、可用性の点において、単一障害時および全種類のサーバの縮退運転時に業務継続可能であることが確認できた。一方で、縮退運転時に縮退割合以上の大幅な性能劣化が発生した（課題 1）。通常時のスループットが 746tps であったのに対し、全種類のサーバを 1 台ずつ停止させた状態では 284tps まで低下し、約 62%の性能劣化が生じた。

性能の点においては、目標とするスループット性能のピーク時 1,000tps が発揮できることを確認した。一方で、1 日当たりの取引数が膨大となった場合に残高取得のレスポンス性能の劣化が発生した（課題 2）。測定データによれば、取引履歴が 0 件の時点では 43ms であったレスポンスが、約 6 万件の時点では 884ms、約 12 万件の時点では 1,724ms、約 18 万件の時点では 2,618ms と、データ量に比例して処理時間が増大した。

セキュリティの点では、非改竄ノードが合意形成ポリシーを満たす状態であれば、一部ノードの改竄があっても業務への影響を防げることが確認された。

### 3.2　実証実験に対する考察

3.1 節で述べた「課題 1：縮退運転時のスループット性能の低下」および「課題 2：取引履歴増加に伴うレスポンス性能の劣化」は、いずれも Fabric のアーキテクチャ特性に深く関連していると考えられる。

課題 1 の縮退時のスループット性能の低下は、Fabric の、データ共有にブロードキャスト方式を用いているという特性に起因すると考えられる。合意形成プロセスにおいて全てのノードと通信するため、一部のノードが障害等で停止している場合でも、そのノードへの通信を試みてタイムアウト待ちが発生し、システム全体の処理が遅延していた。

課題 2 の取引履歴増加に伴うレスポンス性能の劣化は、Fabric の、ネットワークに参加する全てのノードが同一の台帳（World State）を共有するという特性に起因すると考えられる。すべての銀行の取引履歴が単一の DB に蓄積される構造となっていたため、時間の経過とともに全ノードが保持すべきデータ量が大幅に増加し、残高算出のための検索・集計処理の負荷が増大し続けた。

## 4.　提案システム

3 節で述べた、「課題 1：縮退運転時のスループット性能の低下」および「課題 2：取引履歴増加に伴うレスポンス性能の劣化」は、いずれも、合意形成プロセスにおいて全ノードへのブロードキャストを必要とし、全ノードが同一の台帳を共有するという Fabric のアーキテクチャ特性に起因していると考えられる。本節では、これらに対し、異なるアーキテクチャをもつ Corda を採用した小型取引の即時グロス決済を実現する銀行間決済システムを提案する。

### 4.1　提案システムの概要

図 1 に Corda を活用した提案システムの概要を示す。

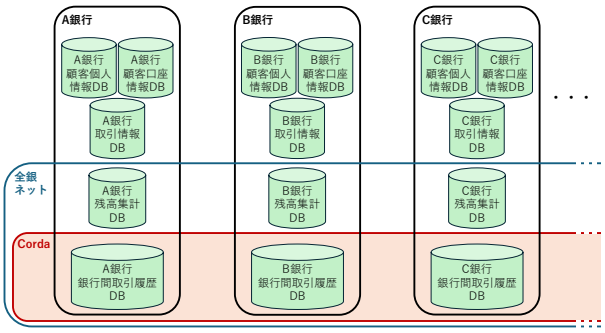



**図 1**　Corda を用いた小口取引即時グロス銀行間決済システム

図 1 のシステムでは、全銀ネットのシステムと各銀行のシステムが接続されている。各銀行は、自行の顧客の個人

© 2026 Information Processing Society of Japan

情報や口座情報、取引情報、銀行残高と自行が関係する銀行間取引履歴の DB を個別に保有している。全銀ネットと各銀行は、Corda ネットワークのノードとして登録され、それぞれが保有する銀行間取引履歴 DB は、Corda 上で Vault とよばれるローカルの DB として管理されている。全銀ネットは Notary として機能し、各銀行間の取引の正当性を検証する役割を担う。

### 4.2 縮退運転時のスループット性能

実証実験における課題 1 は、システムの一部のノードが停止した場合に、システム全体のスループットが約 62%低下するという問題であった。これは、Fabric の、データ共有にブロードキャスト方式を用いているという特性に起因する。合意形成プロセスにおいて全てのノードと通信を行おうとするため、一部のノードが障害等で停止している場合でも、そのノードへの通信を試みてタイムアウト待ちが発生し、システム全体の処理が遅延していた。

これに対して、Corda を用いた本提案システムでは、取引を完了させるために必要な通信は、送金をする銀行、受け取る銀行、および監視ノードの Notary（全銀ネット）との間でのみ発生する。Corda ネットワーク内の取引に無関係な銀行とは通信を行う必要がないため、ある銀行のノードが停止していても、その銀行が関与しない他の銀行間の取引には一切影響を及ぼさない。この特性により、一部のノードに障害が発生しても、無関係な取引の処理速度が低下することを回避でき、システム全体としてのスループット性能を維持することが可能であると考えられる。

### 4.3 取引履歴増加時のレスポンス性能

課題 2 は、取引履歴の件数が増加するにつれて残高取得レスポンスが悪化する（約 18 万件時点で 2 秒以上）という問題であった。これは、Fabric の、ネットワークに参加する全ノードが同一の台帳（World State）を共有するという特性に起因する。実証実験では、すべての銀行の取引履歴が単一の World State に蓄積される構造となっていたため、時間の経過とともに World State のデータ量が増加し、残高取得のための検索・集計処理の負荷が増大し続けた。

これに対して、Corda を用いた本提案システムでは、Corda の Need-to-know の原則に基づき、取引に関係のあるノード間でのみデータを共有する。自銀行が関係していない取引のデータを持つ必要がなくなり、各銀行が保持・処理すべきデータは、自銀行が関与する取引のみに限定される。この特性により、台帳の肥大化を Fabric と比較して大幅に抑制することが可能となり、取引履歴の増加に伴う残高取得のための検索・集計処理の負荷増大を防ぎ、レスポンス性能の維持・改善が見込まれる。

**表 1** 実験環境

| 項目 | 仕様 |
|---|---|
| OS | Windows 11 Home |
| メモリ | 16GB |
| コンテナ基盤 | Docker Desktop（WSL2 バックエンド） |
| Corda バージョン | Corda Community Edition 4.9 |
| Docker イメージ | corda/community:4.9-zulu-openjdk8 |
| JDK | Azul Zulu JDK 8 / 17 |

**表 2** ノード構成

| ノード名 | 役割 | 相当する実体 |
|---|---|---|
| NotaryA | 公証ノード | 全銀ネット |
| BankA | 送金・受取銀行 | 参加銀行 |
| BankB | 送金・受取銀行 | 参加銀行 |
| BankC | 第三者銀行（可用性検証用） | 参加銀行 |

## 5. 提案システムの実装と実験

本節では、4 節で提案した Corda を用いた小口取引即時グロス銀行間決済システムの実現可能性を検証するため、Docker コンテナ上に簡易的な試作システムを構築し、機能要件および非機能要件の観点から実験を行う。機能要件として、銀行間決済の完了確認、残高超過取引の拒否確認、および取引履歴チェックの 3 項目を検証する。非機能要件として、可用性の観点から一部ノード停止時の取引継続性およびスループットへの影響を確認し、性能の観点からスループット性能測定および取引件数増加に伴う残高照会レスポンスの変化を計測する。

### 5.1 実験環境

試作は、ホストマシン上に Docker コンテナを用いて Corda ネットワークを構築し、Windows 11 環境にて実施した。使用したハードウェアおよびソフトウェアの環境を表 1 に示し、ノード構成を表 2 に示す。

### 5.2 実装詳細

本節では実装の詳細を説明する。

#### 5.2.1 環境構築

Corda ネットワークの構築にあたり、まず各ノードの設定ファイル（node.conf）を作成した。node.conf には、ノードの法人名（myLegalName）、P2P 通信アドレス、RPC 接続設定、およびユーザ認証情報を記述した。次に、Network Bootstrapper を用いて各ノードの証明書およびネットワークパラメータを生成した。その後、docker-compose.yml を作成し、各ノードを Docker コンテナとして起動した。

#### 5.2.2 システムの実装

Corda の CorDapp として送金フローおよび残高管理サービスを実装した。実装したクラスの一覧を表 3 に示す。

CashState は残高データを表す State オブジェクトであり、金額（amount）、所有者（owner）、およびユニーク ID

© 2026 Information Processing Society of Japan

表 3　実装した CorDapp クラス一覧

| クラス名 | 役割 |
|---|---|
| CashState | 残高データを表す State オブジェクト |
| CashContract | 取引ルールの定義・検証 |
| IssueCashFlow | 残高の発行フロー |
| TransferCashFlow | 送金フロー |
| BalanceService | 残高集計 DB を管理する CordaService |
| GetBalanceFlow | 残高集計 DB から残高を照会するフロー |

表 4　機能要件の検証結果

| 機能要件 | 検証結果 |
|---|---|
| 銀行間決済の完了 | ○ |
| 残高を超える取引の拒否 | ○ |
| 取引履歴チェック | ○ |

(linearId) から構成される。CashContract は取引ルールを定義し、発行時は入力 State が存在しないこと、送金時は入出力金額の合計が一致することを検証する。IssueCashFlow は指定した金額の CashState を新規に発行するフローである。TransferCashFlow は送金フローであり、Vault 内の UNCONSUMED（未使用）な CashState をページング検索により送金額に達するまで取得し、InputState として消費する。残高が不足している場合は FlowException を発生させて取引を拒否する設計とした。

BalanceService は CordaService として実装した残高管理サービスであり、ノード起動時に自動的に開始される。Observable Vault を用いて CashState の変化を監視し、取引が発生するたびに残高集計 DB をリアルタイムで更新する。具体的には、新たに UNCONSUMED な CashState が追加された場合は残高集計 DB に金額を加算し、CONSUMED になった CashState が発生した場合は残高集計 DB から金額を減算する。GetBalanceFlow は BalanceService が管理する残高集計 DB から残高を照会するフローである。この方式では、残高照会時に Vault からの全件取得が不要となり、取引件数の増加によらず一定の応答時間で残高を取得することができる。

性能測定には、Corda の RPC クライアントを Java で実装した（RpcClient.java）。RpcClient.java は BankA および BankB の RPC ポートに接続し、IssueCashFlow、TransferCashFlow、GetBalanceFlow を startFlowDynamic() により呼び出して逐次実行する。

### 5.3　機能検証

本節では、機能要件として設定した 3 項目の動作確認を行う。検証結果の概要を表 4 に示す。

#### 5.3.1　銀行間決済フローの動作確認

RpcClient.java を用いて BankA に 100 ドルを発行し、TransferCashFlow により BankA から BankB へ 100 ドルの送金を実行した。送金完了後、vaultQuery コマンドを用いて Vault の内容を照会したところ、BankA の Vault では送金前の CashState（100 ドル）が CONSUMED（使用済み）となり、BankB の Vault に新たな CashState（100 ドル）が UNCONSUMED（未使用）として記録されていることを確認した（表 5）。また BankC の Vault には当該取引データが共有されていないことも確認した。これは Corda の Need-to-know の原則に基づくプライバシー保護機能が正常に動作していることを示している。

表 5　送金後の Vault 状態

| ノード | CashState | 状態 |
|---|---|---|
| BankA | 100 ドル | CONSUMED（消費済み） |
| BankB | 100 ドル | UNCONSUMED（未使用） |
| BankC | なし | 記録されない |
| NotaryA | なし | 記録されない |

表 6　可用性要件の検証結果

| 非機能要件（可用性） | 検証結果 |
|---|---|
| 一部ノード停止時に他ノードで取引ができるか | ○ |
| 一部ノード停止時にスループットが変化するか | 変化なし |

#### 5.3.2　残高を超える取引の拒否確認

BankA に 100 ドルを発行後、80 ドルを BankB へ送金し、残高が 20 ドルとなった状態で 200 ドルの送金を試みた。TransferCashFlow は Vault 内の UNCONSUMED な CashState をページング検索して残高を確認し、送金額が残高を超える場合は FlowException を発生させる。実験の結果、以下のエラーメッセージが出力され、残高超過の取引が正常に拒否されることを確認した。

```
Insufficient balance.
Required: 200, Available: 20
```

#### 5.3.3　取引履歴チェックの確認

vaultQuery コマンドを用いて、BankA の Vault に記録された取引履歴を照会した。照会結果には、各取引の amount（金額）、owner（所有者）、recordedTime（記録時刻）、および status（UNCONSUMED/CONSUMED）が含まれており、取引履歴を正確に確認できることを確認した。また、BankA の Vault には BankA が関与する取引のみが記録されており、BankB-BankC 間など無関係な取引は含まれていないことも確認した。

### 5.4　非機能検証

非機能要件である可用性と性能について検証する。

#### 5.4.1　可用性検証

可用性に関する検証結果を表 6 に示す。それぞれについて、以降の節で説明する。

##### 5.4.1.1　一部ノード停止時の取引継続性

BankC を追加した 4 ノード構成（NotaryA、BankA、BankB、BankC）において、以下のコマンドにより BankC を停止した状態で BankA-BankB 間の送金を実行した。

```
docker compose stop bankc
```

© 2026 Information Processing Society of Japan

表 7　スループット性能測定結果（3 回平均）

| 送信件数（件） | 成功件数（件） | 成功率（%） | TPS |
|---|---|---|---|
| 100 | 100 | 100 | 4.96 |
| 500 | 500 | 100 | 4.78 |
| 1,000 | 1,000 | 100 | 4.83 |
| 1,500 | 1,500 | 100 | 4.89 |
| 2,000 | 2,000 | 100 | 4.89 |

BankC 停止後に BankA-BankB 間の送金を実行したところ、取引は正常に完了した。これにより、取引に無関係なノードが停止していても、当該取引の処理には影響が生じないことが確認された。これは Corda の Need-to-know の原則に基づき、BankA-BankB 間の取引において BankC との通信が発生しないためである。

#### 5.4.1.2　一部ノード停止時のスループットへの影響

BankC 停止前後においてスループットを計測したところ、いずれの測定においても約 4.9 TPS を維持しており、スループットに変化がないことを確認した。Fabric では停止ノードへのタイムアウト待ちが発生してスループットが低下したが、Corda では BankC との通信が発生しないためスループットには影響しない。

### 5.4.2　性能検証

次に性能について評価する。

#### 5.4.2.1　スループット性能測定

RpcClient.java を用いて、BankA と BankB の間で交互に送金を行い、100 件から 2,000 件の送金を逐次実行して TPS を計測した。BankA から BankB への送金と BankB から BankA への送金を交互に繰り返すことで、送金のたびに受取側の UNCONSUMED な State が補充され、残高不足が発生しない状況を実現した。各測定を 3 回実施し、その平均値を表 7 に示す。TPS は、最初の取引完了と最後の取引完了の時刻の差分を処理時間として計算した。

TPS は約 4.8〜5.0 の範囲で安定しており、件数が増加しても大きな変動は見られなかった。

#### 5.4.2.2　取引件数増加に伴う残高照会レスポンスの変化

取引件数の増加が残高照会レスポンスに与える影響を測定するため、Observable Vault を用いて CashState の変化をリアルタイムで監視し残高集計 DB を更新する Observable Vault 方式により計測を行った。BalanceService が CashState の追加・消費を検知するたびに残高集計 DB を更新し、GetBalanceFlow により残高集計 DB から残高を取得する際の応答時間を計測した。残高照会は各測定点において 10 回実施し、その平均値を記録した。測定には BankA と BankB の間でランダムな金額（1,000〜10,000 ドル）を往復送金する方式を採用し、送金のたびに送金元に送金額と同額を発行してから送金することで残高不足を防いだ。各測定を 3 回実施し、その平均値を表 8 に示す。

取引件数が増加しても残高照会レスポンスは約 9〜14ms で安定しており、件数増加による影響をほぼ受けないことが確認された。なお、0 件時のレスポンスが 114ms と高い値を示したのは、RPC 接続直後の JVM ウォームアップの影響によるものであり、100 件以降は安定している。

表 8　取引件数増加に伴う残高照会レスポンス（3 回平均）

| 取引件数（件） | 照会時間（ms） | 増加分（ms） |
|---|---|---|
| 0 | 114 | - |
| 100 | 14 | -100 |
| 500 | 13 | -1 |
| 1,000 | 11 | -2 |
| 1,500 | 9 | -2 |
| 2,000 | 9 | +0 |

## 6.　考察

本節では評価実験の結果を考察する。

### 6.1　実証実験との比較

まず全銀ネットの実証実験との比較を述べる。

#### 6.1.1　縮退運転時のスループット性能

実証実験では、全種類のサーバを 1 台ずつ停止させた縮退運転時において、通常時（746 TPS）と比較してスループットが約 62%低下し、284 TPS となった。これは Fabric の合意形成プロセスにおいて全ノードへのブロードキャストが必要なため、停止ノードへの通信でタイムアウト待ちが発生したことに起因する。

これに対して、本研究の実験では BankC を停止した状態で BankA-BankB 間の取引を実行したところ、スループットに変化がないことを確認した（5.4.1.1 節）。これは Corda の Need-to-know の原則に基づき、BankA-BankB 間の取引において BankC との通信が発生しないためである。すなわち、提案システムでは取引に無関係なノードが停止していても、当該取引のスループットに影響がなく、実証実験における課題 1 を解決できることが示された。

ただし、本実験における TPS の絶対値（約 4.9 TPS）は実証実験（746 TPS）と比較して大幅に低い。これは、実証実験が専用サーバー複数台を用いた本番相当の環境であるのに対し、本実験はローカル Docker 環境において全ノードが単一マシン上のコンテナとして CPU・メモリリソースを共有しているためであると考えられる。

#### 6.1.2　取引履歴増加時のレスポンス性能

実証実験では、取引件数が増加するにつれて残高照会レスポンスが悪化し、約 18 万件時点で 2 秒以上となった。これは Fabric の World State に全銀行の取引明細が蓄積される構造に起因し、データ量の増大とともに残高取得のための検索・集計処理の負荷が増大し続けた。

提案システムでは、Observable Vault を用いた残高集計 DB 方式を採用し、取引ごとに BalanceService が CashState の変化を検知して残高集計 DB をリアルタイムで更新することとした。その結果、残高照会時に Vault からの全件取

© 2026 Information Processing Society of Japan

得が不要となり、取引件数に依らず約 9〜14ms で安定した。すなわち、実証実験における課題 2（取引履歴増加に伴うレスポンス性能の劣化）を解決できることが示された。

### 6.2 提案システムの実現可能性

提案システムの機能要件と非機能要件について考察する。

機能要件については、銀行間決済フローの正常完了、残高超過取引の拒否、および取引履歴チェックの全 3 項目が正常に動作することを確認した。特に残高超過チェックについては、TransferCashFlow において Vault 内の UNCONSUMED な State をページング検索して残高を確認し、送金額が残高を超える場合は FlowException を発生させて取引を拒否することを実験により確認した。

非機能要件の可用性の観点では、BankC を停止した状態でも BankA-BankB 間の取引が正常に継続され、スループットに変化がないことを確認した。性能の観点では、Observable Vault 方式による残高照会レスポンスが取引件数の増加によらず約 9〜14ms で安定することを確認した。

以上より、Corda を用いた提案システムが小口取引即時グロス銀行間決済システムとして有効であると考えられる。

### 6.3 課題と留意点

最後に課題と留意点について説明する。

#### 6.3.1 実際のネットワーク環境における動作検証と構成

本実験はローカル Docker 環境における小規模な検証であり、実際の銀行間ネットワーク環境での動作検証が必要である。全銀ネットに参加する 1,000 行以上の金融機関が参加する大規模ネットワークでの性能検証、各ノードへの専用サーバーの割り当て、および冗長化構成の検討が今後の課題として挙げられる。

#### 6.3.2 実証実験のセキュリティ要件への適合性確認

実証実験では、通信暗号化（TLS、IPSec-VPN）、アクセス制御（Firewall）、不正侵入検知（IDS/IPS）、WAF によるアプリケーション脆弱性対策、および ID/パスワード認証によるアプリ不正利用対策が実施された。提案システムにおいても、これらのセキュリティ要件への適合性を確認する必要がある。Corda は TLS によるノード間通信の暗号化および証明書による認証機能を標準で提供しているが、WAF や IDS/IPS などの追加的なセキュリティ対策の実装については別途検討が必要である。

## 7. まとめ

本研究では、全銀ネットが Hyperledger Fabric を用いて実施した銀行間決済プラットフォームの実証実験で明らかとなった 2 つの性能課題、縮退運転時のスループット低下および取引履歴増加に伴う残高取得レスポンスの悪化を解決するため、Corda を用いた小口取引即時グロス銀行間決済システムを提案し、Docker コンテナ上に試作システムを構築して性能評価を行った。

機能検証では、銀行間決済フローの正常完了、残高超過取引の拒否、および取引履歴チェックの全機能が正常に動作することを確認した。

非機能検証では、可用性に関して BankC を停止した状態でも BankA-BankB 間の取引が正常に継続され、スループットに変化がないことを確認し、実証実験における課題 1（縮退運転時のスループット低下）を解決できうることを示した。また、性能に関しては、BankA と BankB が交互に送金を行う方式によるスループット測定において、約 4.8〜5.0 TPS の安定したスループットを確認した。さらに、残高照会レスポンスの測定では、取引件数が増加しても約 9〜14ms で安定した結果が得られ、実証実験における課題 2（取引履歴増加に伴うレスポンス性能の劣化）を解決できうることを示した。

今後の課題としては、実際のネットワーク環境での大規模検証、セキュリティ要件への適合性確認が挙げられる。

**謝辞:** 本研究は JSPS 科研費 JP24K14951 の助成を受けたものです。

### 参考文献

[1] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the thirteenth EuroSys conference*, pages 1–15, 2018.

[2] Richard Gendal Brown, James Carlyle, Ian Grigg, and Mike Hearn. Corda: an introduction. *R3 CEV, August*, 1(15):14, 2016.

[3] Vitalik Buterin. Ethereum white paper: A next-generation smart contract and decentralized application platform. *white paper*, 3(37):2–1, 2014.

[4] James Chapman, Rodney Garratt, Scott Hendry, Andrew McCormack, and Wade McMahon. Project Jasper: Are distributed wholesale payment systems feasible yet? *Financial System*, 59:59, 2017.

[5] Jeffrey Dean and Sanjay Ghemawat. LevelDB github repository. Version 1.2, `https://github.com/google/leveldb/blob/main/doc/index.md` (最終アクセス日：2026 年 1 月 7 日).

[6] Mike Hearn and Richard Gendal Brown. Corda: A distributed ledger. *Corda Technical White Paper*, 2019.

[7] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.

[8] Diego Ongaro and John K. Ousterhout. In search of an understandable consensus algorithm. In Garth Gibson and Nickolai Zeldovich, editors, *Proceedings of the 2014 USENIX Annual Technical Conference, USENIX ATC 2014, Philadelphia, PA, USA, June 19-20, 2014*, pages 305–319. USENIX Association, 2014.

[9] 一般社団法人全国銀行資金決済ネットワークブロックチェーン実証実験タスクフォース. ブロックチェーン技術を活用した新銀行間決済プラットフォーム実証実験報告書. 2019.

© 2026 Information Processing Society of Japan